

Numerical optimization and graphical models

Joanna Pokora

Introduction

This report focuses on the numerical optimization methods and graphical models. First, we define and compare gradient descent, cyclic coordinate gradient descent and Newton's method. In the second part, we estimate the precision matrix using several methods under different true conditional dependency structures.

1. Numerical optimization

We consider the following function

$$f(x, y, z) = (x - 1)^4 - 4(x - 1)^3 + 4(x - 1)^2 + (y + 1)^4 - 4(y + 1)^3 + 4(y + 1)^2 \\ + (z - 2)^4 - 4(z - 2)^3 + 4(z - 2)^2 + (x - 1)(y + 1) - (y + 1)(z - 2).$$

As was shown in class, $f(1, -1, 2) = 0$ is a local minimum. We now aim to find this minimum point using numerical optimization methods.

1.1 Gradient descent

Gradient descent is an iterative algorithm that updates all coordinates at each step. The update at $k + 1$ iteration is given by

$$\begin{pmatrix} x^{k+1} \\ y^{k+1} \\ z^{k+1} \end{pmatrix} = \begin{pmatrix} x^k \\ y^k \\ z^k \end{pmatrix} - \eta \nabla f(x^k, y^k, z^k),$$

where η is the step size. Thus, we need to calculate the gradient of the function f . In the considered scenario it takes the form

$$\nabla f(x, y, z) = \begin{pmatrix} \frac{\partial f}{\partial x} \\ \frac{\partial f}{\partial y} \\ \frac{\partial f}{\partial z} \end{pmatrix} = \begin{pmatrix} 4(x - 1)^3 - 12(x - 1)^2 + 8(x - 1) + y + 1 \\ 4(y + 1)^3 - 12(y + 1)^2 + 8(y + 1) + x - z + 1 \\ 4(z - 2)^3 - 12(z - 2)^2 + 8(z - 2) - y - 1 \end{pmatrix}.$$

1.2 Cyclic coordinate gradient descent

Cyclic coordinate gradient descent is a modification of the previous method. In each step we update only one coordinate at a time. The coordinates are selected and updated cyclically. The full $k + 1$ cycle is given by

$$x_{k+1} = x_k - \eta_x \frac{\partial f}{\partial x}(x^k, y^k, z^k) \\ y_{k+1} = y_k - \eta_y \frac{\partial f}{\partial y}(x^k, y^k, z^k) \\ z_{k+1} = z_k - \eta_z \frac{\partial f}{\partial z}(x^k, y^k, z^k).$$

Here, we use partial derivatives from the gradient calculated for the previous method.

1.3 Newton's method

The $k + 1$ iteration of Newton's algorithm takes the form

$$\begin{pmatrix} x^{k+1} \\ y^{k+1} \\ z^{k+1} \end{pmatrix} = \begin{pmatrix} x^k \\ y^k \\ z^k \end{pmatrix} + d^k,$$

where d^k is the direction calculated as

$$d^k = -\nabla^2 f^{-1}(x^k, y^k, z^k) \nabla f(x^k, y^k, z^k).$$

Since we have already calculated the gradient, we only need to find the Hessian matrix. It is given by

$$\nabla^2 f(x, y, z) = \begin{pmatrix} 12x^2 - 48x + 44 & 1 & 0 \\ 1 & 12y^2 - 4 & -1 \\ 0 & -1 & 12z^2 - 72z + 104 \end{pmatrix}.$$

In each iteration we compute $\nabla^2 f(x^k, y^k, z^k)$ and inverse it.

1.4 Comparison of the numerical optimization methods

We now run all introduced algorithms with starting point $(x^0, y^0, z^0) = (1.2, -0.8, 2.1)$ until

$$f(x^k, y^k, z^k) - f(1, -1, 2) < 0.01.$$

Additionally, we set the step sizes in both gradient descend methods to $\eta = \eta_x = \eta_y = \eta_z = 0.05$.

The following table presents the number of iterations, final iterate and the final value of the objective function for all algorithms.

Method	Number of iterations	Final iterate	Final value
gradient descent	5	(1.015, -0.978, 2.019)	0.004
cyclic coordinate gradient descent	5 (full cycles)	(1.015, -0.974, 2.016)	0.004
Newton's	2	(0.992, -1.012, 1.997)	0.001

The closest to 0 final value was obtained by Newton's method. It also converged the fastest, requiring only two iterations. While this algorithm provides high accuracy and fast convergence near the solution, it requires calculating the Hessian matrix, which is not always straightforward.

Both gradient methods reached a slightly larger final value and required three more iterations than Newton's method.

In this simulation, gradient descent obtained the minimum in relatively few iterations. However, it is sensitive to the choice of the step size and may end up zig-zagging if η is too large.

Cyclic coordinate gradient descent yielded very similar results to the regular gradient descent. Since only one coordinate is updated at a time, the performance of this method depends on the scaling of the coordinates and coupling between the variables.

2. Graphical models

We consider two settings:

1. a block diagonal covariance matrix

$$\Sigma = \begin{pmatrix} \Sigma^{(1)} & 0 \\ 0 & \Sigma^{(2)} \end{pmatrix},$$

where each block takes the form

$$\Sigma_{50 \times 50}^{(k)} = 0.2 \mathbb{I}_{50 \times 50} + 0.8 \mathbf{1} \mathbf{1}^T, \quad k \in \{1, 2\},$$

with $\mathbb{I}_{50 \times 50}$ denoting the identity matrix and $\mathbf{1} \in \mathbb{R}^{50}$ denoting the column vector of ones;

2. a hub precision matrix

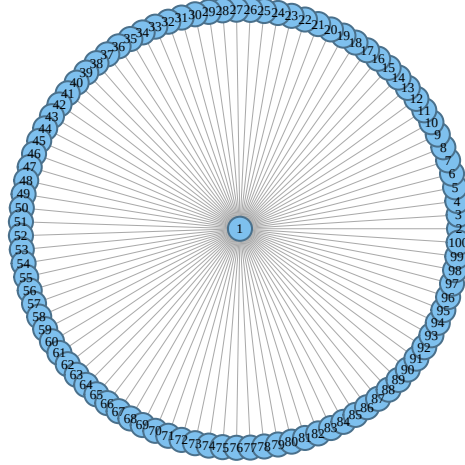
$$\Omega = \begin{pmatrix} 60 & -0.7 & -0.7 & \cdots & -0.7 \\ -0.7 & 1 & 0 & \cdots & 0 \\ -0.7 & 0 & 1 & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ -0.7 & 0 & 0 & \cdots & 1 \end{pmatrix}.$$

The following graphs, based on the precision matrices, present the conditional dependence between the variables in both settings.

Block matrix



Hub matrix



From the block diagonal covariance matrix we obtain a block diagonal precision matrix, visible as two clusters in the graph. The second graph illustrates the hub structure with conditional dependence between the first variable and all 99 other variables.

Next, for each setting, we simulate $X_1, \dots, X_{300} \sim N_{100}(0, \Sigma)$ independent observations. We want to compare the following estimators of the precision matrix.

- a. The maximum likelihood estimator (MLE)

$$\hat{\Omega}_{MLE} = S^{-1} = \left(\frac{1}{n} \sum_{i=1}^n (x_i - \bar{x})(x_i - \bar{x})^T \right)^{-1},$$

where x_i is the column vector representing the i -th observation and $\bar{x}$ is the column vector of variable means.

- b. The inverse of the Ledoit-Wolf shrinkage covariance estimator

$$\hat{\Omega}_{LW} = \hat{\Sigma}_{LW}^{-1} = ((1 - \lambda)S + \lambda \mathbb{I})^{-1},$$

where

$$\lambda = \frac{\frac{1}{n^2} \sum_{k=1}^n \|Y_k - S\|_F^2}{\|S - T\|_F^2}, \quad Y_k = (x_k - \bar{x})(x_k - \bar{x})^T, \quad k \in \{1, \dots, n\}.$$

Here, we use the `linearShrinkLWEst()` function.

- c. The graphical LASSO

$$\hat{\Omega}_{gLASSO} = \arg \max_{\Omega \in \text{Sym}_+^p} \{ \log \det(\Omega) - \text{tr}(S\Omega) - \rho \|\Omega\|_1 \},$$

where $\|\Omega\|_1 = \sum_{i < j} |\omega_{ij}|$ is the L1 norm.

- d. The graphical SLOPE - let $\omega = [\omega_1, \dots, \omega_m]$ be the vectorized upper triangle of Ω . Then

$$\hat{\Omega}_{gSLOPE} = \arg \max_{\Omega \in \text{Sym}_+^p} \left\{ \log \det(\Omega) - \text{tr}(\Omega S) - \rho \sum_{i=1}^m \lambda_i |\omega|_{(i)} \right\},$$

where $|\omega|_{(1)} \geq \dots \geq |\omega|_{(m)}$ and $\lambda_1 \geq \dots \geq \lambda_m \geq 0$. We use λ sequence controlling FDR at level $\alpha = 0.2$, defined as

$$\lambda_k^{BH} = \frac{t_{n-2} \left(1 - \frac{\alpha k}{2m}\right)}{\sqrt{n-2 + t_{n-2}^2 \left(1 - \frac{\alpha k}{2m}\right)}}, \quad k \in \{1, \dots, m\}.$$

- e. The partial correlation graphical LASSO - let R be the following matrix:

$$R_{ij} = \frac{\Omega_{ij}}{\sqrt{\Omega_{ii}\Omega_{jj}}} = -\rho_{ij}, \quad R_{ii} = 1,$$

where ρ_{ij} is the partial correlation. Let $D^2 = \text{diag}(\Omega)$. Then pcgLASSO estimator is given by

$$\hat{\Omega}_{pcgLASSO} = \hat{D} \hat{R} \hat{D},$$

$$(\hat{R}, \hat{D}) \in \arg \max_{R, D} \{ \log \det(DRD) - \text{tr}(SDRD) - \rho \|R\|_{1, \text{off}} - 2\alpha \log \det(D) \},$$

where $\alpha < 1$ takes the largest possible value for which the solution exists (default for the `pcglasso()` function).

Since graphical LASSO, graphical SLOPE and pcgLASSO require specifying the tuning parameter ρ , we choose its value so that the estimated precision matrix would minimize

$$\text{MSE}(\hat{\Omega}) = \frac{1}{p^2} \|\hat{\Omega} - \Omega\|_F^2.$$

2.1 One sample

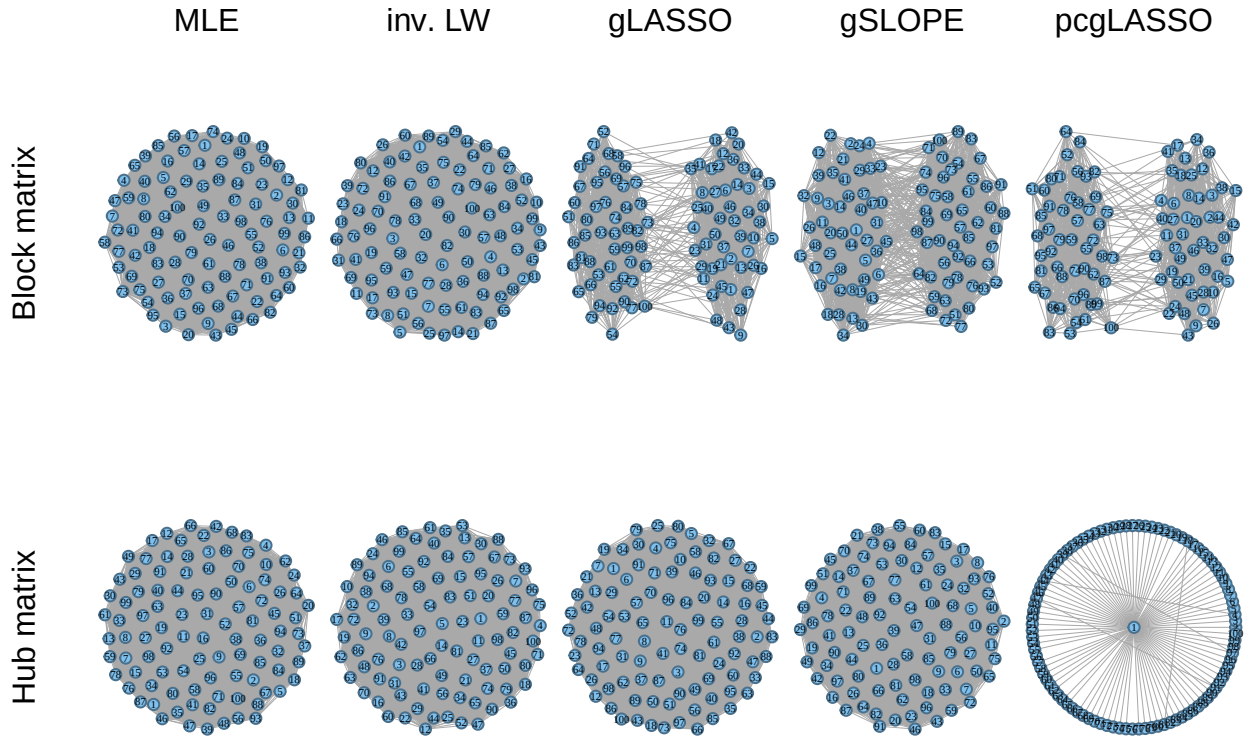
First, we summarize the selected optimal tuning parameters.

	gLASSO	gSLOPE	pcgLASSO
Block matrix	0.034	0.22	0.2
Hub matrix	0.000	0.00	0.2

For the gSLOPE and gLASSO algorithms, the selected tuning parameters were zero when estimating the hub matrix, which resulted in no penalization. Since almost all off-diagonal elements of the hub matrix equal zero, the penalty would have to shrink them accordingly. However, this introduces a bias in the hub estimation. In the considered scenario, this leads to the higher MSE, which results in complete drop of the penalization.

For pcgLASSO, the optimal parameter was the same under both settings.

The following graphs show the conditional dependency structure from the precision matrices estimated using the introduced methods under both settings.



Both the maximum likelihood and inverse Ledoit-Wolf estimators failed to reconstruct the original structures. These methods do not account for sparsity, so they return matrices with no zeros.

In case of the block matrix, clustering is visible for penalized methods, suggesting that they produce a significantly better fit. However, for the hub matrix, only pcgLASSO successfully recovered the original structure.

Next table presents the mean squared errors of the estimated precision matrices.

	MLE	inv. LW	gLASSO	gSLOPE	pcgLASSO
Block matrix	0.331	0.190	0.011	0.006	0.019
Hub matrix	0.178	0.352	0.122	0.122	0.003

For the block matrix, the lowest mean squared error was achieved by gSLOPE. Because the sorted L1 norm penalizes larger coefficients more strictly, it shrinks the similar absolute values toward each other, resulting in clustering. Consequently, gSLOPE achieves good results with true block structure. Moreover, both gLASSO and pcgLASSO also obtained quite low MSE under this setting.

In case of the hub matrix, the lowest mean squared error was achieved by pcgLASSO. By penalizing the partial (inverse) correlations rather than the absolute precision matrix elements (inverse covariances), pcgLASSO does not depend on the variable scaling. Furthermore, unlike for any other method, the MSE for pcgLASSO was lower for the hub precision matrix than for the block diagonal matrix.

The maximum likelihood and inverse Ledoit-Wolf estimators had the highest errors, which along with failure to recover structure demonstrates that they are not suitable estimation method in the analysed scenarios, as they do not account for sparsity or specific structure.

The number of false positive and false negative errors from the upper triangle is presented in the table below.

	MLE	inv. LW	gLASSO	gSLOPE	pcgLASSO
Block matrix					
false positive	2500	2500	80	213	69
false negative	0	0	1164	662	1294
Hub matrix					
false positive	4851	4851	4849	4849	6
false negative	0	0	0	0	5

In both settings, the matrices estimated using the maximum likelihood and inverse Ledoit-Wolf methods were fully dense with no zeros.

In case of the block matrix, both gLASSO and pcgLASSO identified relatively few signals. Although gSLOPE yielded a higher number of false positives than these two methods, it halved the number of false negative errors. This resulted in the lower mean squared error, as shown in the previous table.

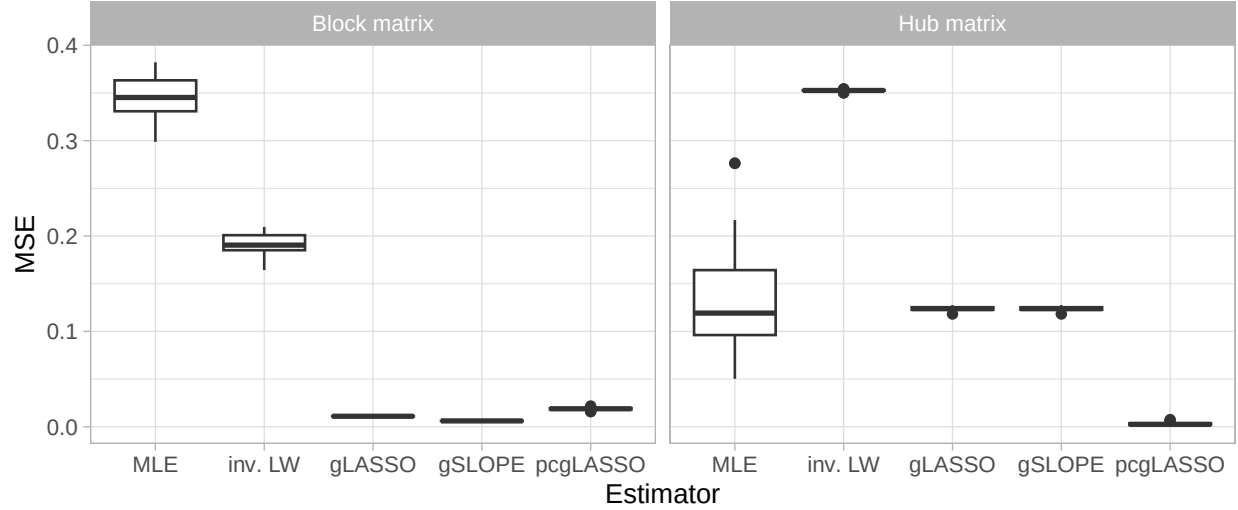
For the hub matrix, because the selected tuning parameters for gLASSO and gSLOPE resulted in no penalization, they identified almost all elements as non-zeros. In contrast, pcgLASSO was very successful in this setting, with very few false positive and false negative errors.

2.2 Repeated experiment

Now, we repeat the simulation 25 times and perform the same estimation steps. First, we plot the estimated optimal parameters for gLASSO, gSLOPE and pcgLASSO under both settings.

Since the number of estimated matrices is much higher in this scenario, we do not investigate them through graphs. Instead, we focus on the MSE and error counts.

First, we examine the mean squared errors under different true structures using various estimation methods. They are visualized on the boxplots and the means over 25 replications are presented in the table below.



	MLE	inv. LW	gLASSO	gSLOPE	pcgLASSO
Block matrix	0.344	0.191	0.011	0.006	0.019
Hub matrix	0.136	0.353	0.124	0.124	0.003

The maximum likelihood estimator yielded the most dispersed MSE values, especially under the hub matrix setting. Overall, the relationships between the MSE values for different methods remain the same as in the single sample case described before. They confirm that gSLOPE performs the best with the block matrix, while pcgLASSO is the best pick with the hub matrix.

Since the counts of both types of errors were very close across replications, we provide only a table containing the average numbers of the false positive and false negative errors from the upper triangle of the estimated matrices.

	MLE	inv. LW	gLASSO	gSLOPE	pcgLASSO
Block matrix					
false positive	2500	2500	107	226	81
false negative	0	0	1180	653	1299
Hub matrix					
false positive	4851	4851	4848	4848	403
false negative	0	0	0	0	1

Once again, the results of the repeated experiment confirm the previous observations.

Although the average number of false positive discoveries under the hub setting for pcgLASSO is larger than in the single sample case, it remains much lower than that of the other methods. Thus, pcgLASSO consistently recovers the true structure most accurately in this case.

For the diagonal block matrix, gSLOPE once again yielded more false positives than the gLASSO and pcgLASSO methods, however it lowered the average count of false negative errors, which ultimately improved the overall structure estimation.

Summary

In this report we first implemented and compared three iterative algorithms: gradient descend, cyclic coordinate gradient descend and Newton's method. In the analysed scenario, both the lowest number of iterations

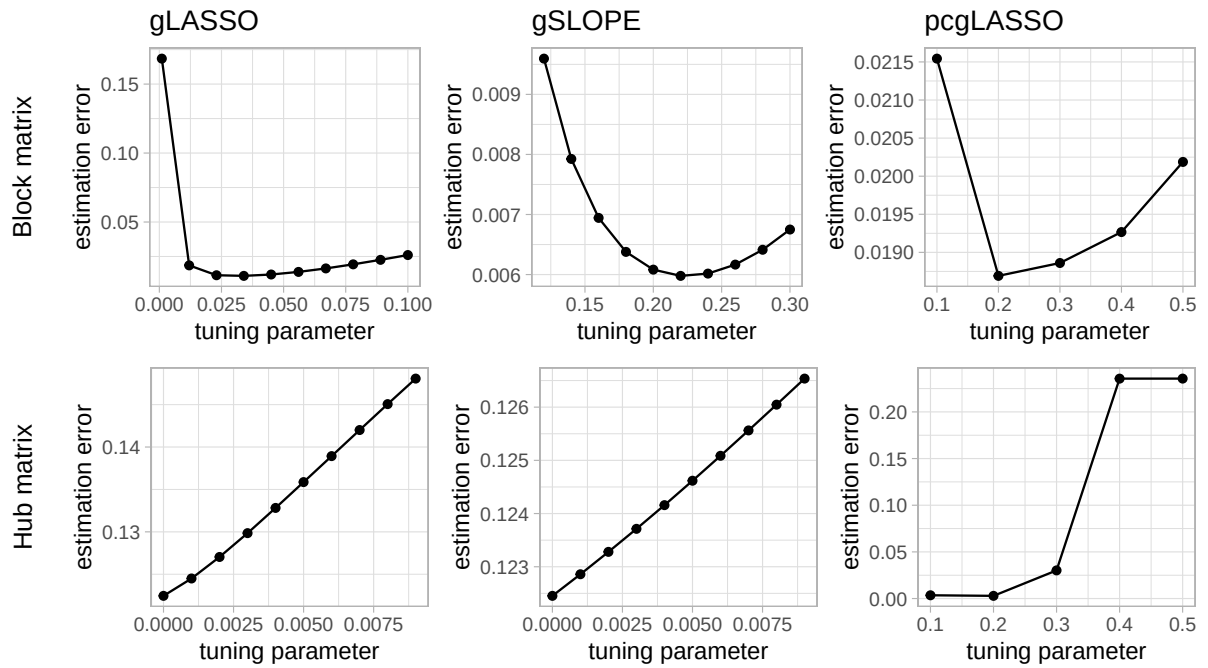
and the most accurate results were obtained by Newton's method. The gradient-based algorithms required three more iterations and yielded final values slightly further from zero. Their great asset is that, unlike Newton's method, they do not require calculating the second derivative, which can be difficult.

The second part focused on the precision matrix estimation under different true conditional dependency structures using various methods. Both the one sample experiment and the results from the 25 replications showed that the most successful method for the block diagonal matrix is gSLOPE, due to its strong clustering properties. For the hub matrix, the most accurate method turned out to be pcgLASSO, as it does not depend on the scaling of the variables.

Appendix - tuning parameter selection

For the gSLOPE model, only one tuning parameter, scaling the entire penalty sequence, was estimated. Additionally, in `gslope_new()` function we set `rho=1`.

The plots below present the searching grids in a one sample case.



In the repeated experiment, the tuning parameters were optimized separately for each simulated data matrix, using the same grids as in the one sample case. The following plot presents the obtained values.

